

Entity

Entity層には、アプリの中心となる型や定数、業務概念を置く。

User

ユーザー本人の識別情報・役割・利用状態・Access Token無効化。
ログインや認証必須の機能や利用はStatusから判定する。

UserRole

値	内容
<code>user</code>	一般ユーザー
<code>admin</code>	管理者

UserStatus

値	内容
<code>active</code>	有効ユーザーでログインできる
<code>suspended</code>	停止中でログインできない

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	ユーザー作成時にDBが発行し、発行後は変更しない。 動画、 <code>Refresh Token</code> 、いいね、保存などからユーザーを参照するために使用
<code>Name</code>	<code>string</code>	画面上で投稿者名として表示する名前。会員登録時に前後の空白を除去し、1~50文字で保存する。
<code>Email</code>	<code>string</code>	ログインとユーザー識別に使用するメールアドレス。 会員登録時に前後の空白を除去し、小文字へ統一して保存する。 同じ正規化済みメールアドレスは重複登録できない
<code>PasswordHash</code>	<code>string</code>	会員登録時にパスワードをハッシュ化した結果を保存する。 ログイン時は入力されたパスワードと照合する。 平文パスワードは <code>Entity</code> 、 <code>DB</code> 、ログへ保存しない

フィールド	型	仕様
<code>Role</code>	<code>UserRole</code>	ユーザーの権限区分。通常登録時は <code>user</code> を設定し、管理者アカウントだけ <code>admin</code> を設定する。 管理者APIの利用可否や、利用停止対象かどうかの判定に使用
<code>Status</code>	<code>UserStatus</code>	ユーザーがサービスを利用できる状態かを表す。 登録時は <code>active</code> を設定する。 管理者による利用停止で <code>suspended</code> へ変更し、利用再開で <code>active</code> へ戻す
<code>TokenVersion</code>	<code>uint64</code>	発行済み <code>Access Token</code> をまとめて無効化するための番号。 初期値は0。 <code>Access Token</code> 発行時に <code>JWT</code> へ同じ値を含め、認証時に <code>JWT</code> 内の値と現在の <code>User.TokenVersion</code> を比較する。 値が一致しない <code>Access Token</code> は無効として拒否する
<code>CreatedAt</code>	<code>time.Time</code>	ユーザーを作成した日時。 会員登録時に設定し、その後は変更しない。
<code>UpdatedAt</code>	<code>time.Time</code>	ユーザー情報または認証状態を最後に変更した日時。 名前変更、利用停止、利用再開、 <code>TokenVersion</code> 更新などの際に更新する

生成方法

Userは会員登録時または管理者アカウント作成時に生成する。構造体リテラルから任意の状態でも生成せず、生成関数を通して初期状態を確定する。

メソッド	引数	生成する状態	失敗条件
<code>NewUser(name, email, passwordHash string, role UserRole, now time.Time) (*User, error)</code>	正規化済み名前、メールアドレス、ハッシュ済みパスワード、役割、作成日時	Statusを <code>active</code> 、TokenVersionを0、CreatedAtとUpdatedAtを <code>now</code> へ設定する	名前、メール、PasswordHash、Roleが不正

不変条件

項目	条件
<code>ID</code>	作成後には変更しない
<code>Name</code>	前後の空白を除去した後、1～50文字
<code>Email</code>	正規化後254文字以内。同じ値を重複保存しない

項目	条件
PasswordHash	平文パスワードを保持しない
Role	user または admin 以外を許可しない
Status	active または suspended 以外を許可しない
管理者	利用停止対象にしない⇒ suspended へ変更しない
suspended	ログイン、Token 再発行、認証必須機能を利用できない
TokenVersion	無効化操作のたびに1増加し、減少させない

操作

このメソッドはメモリ上の状態だけ変更し、DB保存・Token失効・動画の非公開・監査ログの作成はUsecaseが行う

停止理由と再開理由はUserへ重複保存せず、改ざん防止のためAdminAuditLogへ保存する

現在の停止理由が必要な場合は、対象ユーザーの最新監査ログを取得する

メソッド	事前の条件	状態変更	失敗条件・結果
IsActive() bool	なし	なし	Statusが active なら true、suspended なら false を返す。パスワードやTokenの正当性までは判定しない
IsAdmin() bool	なし	なし	Roleが admin なら true、user なら false を返す。管理者APIを実行できるかの最終判定はUsecaseまたはMiddlewareが行う
Suspend(now time.Time) error	Role が user かつ Status が active	Statusを suspended へ変更する。 TokenVersion を1増加し、既存 Access Token を無効化する。UpdatedAt へ now を設定する	Roleが admin の場合、または既に suspended の場合は状態を変更せずエラーを返す
Resume(now time.Time) error	Role が user かつ Status が	Status を active へ変更し、UpdatedAt	Roleが admin の場合、または既に

メソッド	事前の条件	状態変更	失敗条件・結果
	<code>suspended</code>	へ <code>now</code> を設定する。 <code>TokenVersion</code> は変更しない。利用停止時に <code>hidden</code> になった動画も自動公開しない	<code>active</code> の場合は状態を変更せずエラーを返す
<code>InvalidateAccessTokens(now time.Time)</code>	なし	<code>TokenVersion</code> を1増加し、 <code>UpdatedAt</code> へ <code>now</code> を設定する	<code>JWT</code> の署名確認や <code>Refresh Token</code> の失効は行わない。 <code>DB</code> 保存は <code>Usecase</code> と <code>Repository</code> が行う

RefreshToken

RefreshTokenの使用状態、失効状態、有効期限、Tokenのローテーション系列を管理する。

Entityが保持するのはハッシュ値だけであり、ブラウザへ渡す平文Tokenは保持しない。

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	<code>Refresh Token</code> レコードを一意に識別する番号。 <code>DB</code> 作成時に発行し、ローテーション後の <code>RefreshToken</code> を <code>ReplacedByID</code> から参照するために使用する
<code>UserID</code>	<code>uint64</code>	<code>Refresh Token</code> を発行したユーザーの <code>ID</code> 。 <code>Token</code> 再発行時に対象 <code>User</code> を取得し、 <code>Status</code> と <code>TokenVersion</code> を確認するために使用する
<code>TokenHash</code>	<code>string</code>	ブラウザへ渡した平文 <code>Refresh Token</code> をハッシュ化した値。 <code>Cookie</code> から受け取った <code>Token</code> を同じ方式でハッシュ化し、 <code>DB</code> 検索または照合に使用する。平文 <code>Token</code> は保存しない
<code>FamilyID</code>	<code>string</code>	最初のログインから継続する <code>Refresh Token</code> 系列を識別するランダムな値。 <code>Token</code> をローテーションしても同じ値を引き継ぐ。 使用済み <code>Token</code> の再利用を検知した場合、この値が同じ <code>Token</code> をすべて失効する

フィールド	型	仕様
ReplacedByID	*uint64	この Token を使用して新しく発行した Refresh Token の ID 。未使用 Token では nil 。ローテーション成功時に RefreshToken の ID を設定する
ExpiresAt	time.Time	Refresh Token を利用できる期限。現在時刻がこの日時以上の場合は期限切れとして再発行を拒否する
UsedAt	*time.Time	この Token を使って Access Token と次の Refresh Token を再発行した日時。未使用の場合は nil 。値が設定済みの Token が再送された場合は、Token 再利用として扱う
RevokedAt	*time.Time	ログアウト、Token 再利用検知、ユーザー利用停止などで無効化した日時。有効な Token では nil 。値が設定済みの場合は期限内でも利用できない
CreatedAt	time.Time	Refresh Token レコードを発行した日時。ログインまたは Token ローターション時に設定し、後から変更しない

生成方法

Refresh Tokenレコードは、ログイン成功時またはRefresh Tokenローテーション時に生成する。

平文Refresh TokenはEntityへ渡さず、ハッシュ化済みの値だけを渡す。

メソッド	引数	生成する状態	失敗条件
NewRefreshToken(userID uint64, tokenHash, familyID string, expiresAt, now time.Time) (*RefreshToken, error)	ユーザーID、Token Hash、系列ID、有効期限、発行日時	UsedAt 、RevokedAt 、ReplacedByIDを nil として生成する	UserID が0、TokenHash が空、FamilyID が空、ExpiresAt が now 以下

不変条件

項目	条件
TokenHash	平文Refresh Tokenを保存しない
FamilyID	ローテーション後のTokenへ同じ値を引き継ぐ
UsedAt	一度設定した後は元に戻さない
ReplacedByID	UsedAt を設定する場合に新しく発行したRefresh TokenのID も設定する
RevokedAt	一度失効した Token を有効状態へ戻さない
ExpiresAt	現在時刻以上でなければ利用できない
再利用検知	UsedAt != nil の Token が送信された場合、同じ FamilyID の Token をすべて失効する

操作

メソッド	事前条件	状態変更	失敗条件・結果
<code>IsExpired(now time.Time)</code> <code>bool</code>	なし	なし	<code>now >= ExpiresAt</code> の場合に <code>true</code> を返す
<code>IsUsable(now time.Time)</code> <code>bool</code>	なし	なし	<code>UsedAt</code> と <code>RevokedAt</code> が <code>nil</code> で、かつ期限内の場合だけ <code>true</code> を返す
<code>MarkUsed(replacedByID uint64, now time.Time)</code> <code>error</code>	未使用、未失効、期限内、 <code>RefreshToken</code> 作成済み	<code>UsedAt</code> へ <code>now</code> 、 <code>ReplacedByID</code> へ新しく発行した <code>RefreshToken</code> のIDを設定する	使用済み、失効済み、期限切れ、新しく発行した <code>RefreshToken</code> のIDが0の場合は状態を変更せずエラー
<code>Revoke(now time.Time)</code>	なし	<code>RevokedAt</code> が <code>nil</code> の場合だけ <code>now</code> を設定する	既に失効済みの場合は、値を変更せず成功扱いとし、ログアウト再送の冪等性を保つ

`RefreshToken Entity` は1件の `Token` 状態だけを変更する。
 同じ `FamilyID` の一括失効、`User.TokenVersion` の増加、`DB` トランザクションは `Usecase` が行う。

Refresh Tokenの生成・Hash・検索

Refresh Tokenの平文は、認証サービスが暗号学的に安全な乱数から生成する。Entityは平文Tokenを保持しない。

Tokenの扱い

処理	仕様
平文Token生成	暗号学的に安全な乱数を使用する
ブラウザ保存	HttpOnly、Secure、SameSite属性付きCookie
DB保存	平文ではなく決定的なHash値を保存する
Token検索	Cookieの平文Tokenを同じ方式でHash化し、TokenHashで検索する
ログ	平文Token、Cookie値、完全なTokenHashを出力しない
URL	Refresh TokenをURLやQuery Parameterへ含めない

Hash方式

Refresh Tokenは、DB検索可能な決定的Hashを使用する。

パスワード用のbcryptやArgon2は、毎回異なるHashを生成するため、TokenHashによる直接検索には使用しない。

項目	仕様
方式	HMAC-SHA-256
Key	環境変数で管理する専用秘密鍵
入力	平文Refresh Token
出力	出力は小文字の16進数64文字へ統一する。
DB検索	<code>token_hash = ?</code>
UNIQUE	TokenHashへUNIQUE制約
Key漏洩対策	JWT秘密鍵とは別の鍵を使用する

フィールド	仕様
<code>KeyHash</code>	HMAC-SHA-256で生成し、小文字16進数64文字で保存する
<code>RequestHash</code>	SHA-256で生成し、小文字16進数64文字で保存する

検索手順

1. CookieからRefresh Tokenを取得
2. HMAC-SHA-256でTokenHashを生成
3. TokenHashでRefreshTokenを検索
4. UsedAt、RevokedAt、ExpiresAtを確認
5. User.Statusを確認
6. ローテーション処理

再利用検知

条件	処理
UsedAtがnil	通常のローテーション処理
UsedAtが設定済み	再利用検知
RevokedAtが設定済み	再発行拒否
<code>now >= ExpiresAt</code>	期限切れとして拒否

CategoryCode

動画投稿と動画検索で使えるカテゴリーを定義

値	表示名	内容
brewing	抽出	ハンドドリップ、エスプレッソなど
roasting	焙煎	焙煎方法、焙煎技術
latte_art	ラテアート	ラテアート技術
beans	コーヒー豆	豆の紹介、レビュー
equipment	器具	ミル、ドリッパーなど

操作

メソッド	事前条件	状態変更	結果
Valid() bool	なし	なし	定義済みカテゴリーの場合だけ true
DisplayName() string	定義済みカテゴリー	なし	対応する表示名を返す

Video

投稿者、タイトル、説明、カテゴリー、アップロード期限、動画処理状態、公開状態、論理削除状態を管理。

VideoProcessingStatus

値	内容
uploading	動画アップロード中
expired	アップロード期限切れ
uploaded	アップロード完了
processing	動画検証、変換、サムネイル生成中
ready	動画処理完了
failed	動画処理失敗

VideoPublishStatus

値	内容
private	一般公開されていない
published	一般公開されている
hidden	管理者によって非公開になっている

VideoContainer

値	内容
mp4	MP4形式
mov	MOV形式

フィールド

フィールド	型	仕様
ID	uint64	動画投稿を一意に識別する番号。 動画投稿開始時に DB が発行し、発行後は変更しない。 動画処理ジョブ、いいね、保存、監査ログなどから対象動画を参照するために使用する
UserID	uint64	この動画を投稿したユーザーの ID。 動画投稿開始時にログインユーザー ID を設定する。 投稿者本人による非公開、再公開、削除の所有権確認に使用する
Category	CategoryCode	投稿時に選択されたカテゴリーコード。 brewing、roasting、latte_art、beans、equipment のいずれかを保存し、カテゴリー検索と画面表示に使用する
Title	string	リール画面、検索結果、動画詳細へ表示するタイトル。 投稿開始時に前後の空白を除去し、1~100文字で保存する。
Description	string	動画内容の説明。投稿開始時に保存し、1000文字以内とする。 未入力の場合は空文字を保存する。
OriginalObjectKey	string	元動画をオブジェクトストレージ内で特定する内部キー。動画投稿開始時にサーバーが生成する。ユーザーが指定した元ファイル名、タイトル、説明をそのまま保存先キーとして使用しない

フィールド	型	仕様
<code>UploadExpiresAt</code>	<code>time.Time</code>	署名付き <code>URL</code> を使用して元動画をアップロードできる期限。投稿開始時に設定し、アップロード完了通知時に期限内かを判定する。期限を過ぎた場合は処理状態を <code>expired</code> へ変更する
<code>SourceMeta</code>	<code>*SourceVideoMeta</code>	FFprobeによる入力動画の実ファイル検証に成功した後、その解析結果を保存する。アップロード中、検証前、検証失敗時は <code>nil</code> とする
<code>OutputMeta</code>	<code>*OutputVideoMeta</code>	動画変換とサムネイル生成の両方が成功した後、生成物の情報を保存する。処理前、変換失敗、サムネイル生成失敗時は <code>nil</code> とする
<code>ProcessingStatus</code>	<code>VideoProcessingStatus</code>	動画投稿開始から処理完了までの進行状態。作成時は <code>uploading</code> とし、許可された状態遷移メソッドを通してのみ変更する
<code>PublishStatus</code>	<code>VideoPublishStatus</code>	一般利用者へ動画を表示できるかを表す状態。作成時は <code>private</code> とする。動画処理成功時に公開条件を満たせば <code>published</code> 、管理者が非公開にした場合は <code>hidden</code> へ変更する
<code>CreatedAt</code>	<code>time.Time</code>	動画投稿レコードを作成した日時。動画投稿開始時に設定し、その後は変更しない
<code>UpdatedAt</code>	<code>time.Time</code>	タイトル、説明、処理状態、公開状態などを最後に変更した日時。Entityの状態変更メソッドを実行した際に更新する
<code>DeletedAt</code>	<code>*time.Time</code>	投稿者が動画を削除した日時。未削除の場合は <code>nil</code> とする。値が設定された動画は、リール、検索、詳細、保存一覧、いいね対象へ表示しない

生成方法

Videoは動画投稿開始時に生成する。

動画処理済みや公開済みの状態を外部から指定できないようにする。

メソッド	引数	生成する状態	失敗条件
<code>NewVideo(userID uint64, category CategoryCode, title, description, originalObjectKey string, uploadExpiresAt, now time.Time) (*Video, error)</code>	投稿者ID、カテゴリー、タイトル、説明、元動画キー、アップロード期限、作成日時	<code>uploading / private</code> 、 <code>SourceMeta</code> 、 <code>OutputMeta</code> 、 <code>DeletedAt</code> は <code>nil</code>	UserID、Category、Title、Description、ObjectKey、期限が不正

初期状態

項目	初期値
<code>ProcessingStatus</code>	<code>uploading</code>
<code>PublishStatus</code>	<code>private</code>
<code>SourceMeta</code>	<code>nil</code>
<code>OutputMeta</code>	<code>nil</code>
<code>DeletedAt</code>	<code>nil</code>

不変条件

項目	条件
<code>UserID</code>	0以外
<code>Category</code>	定義済みの <code>CategoryCode</code>
<code>Title</code>	trim後1～100文字
<code>Description</code>	1000文字以内
<code>OriginalObjectKey</code>	空文字ではない
<code>UploadExpiresAt</code>	作成時刻より後
公開条件	<code>ProcessingStatus = ready</code> 、 <code>PublishStatus = published</code> 、 <code>DeletedAt = nil</code>
<code>hidden</code>	管理者だけが設定・解除できる
削除済み	閲覧、検索、いいね、保存、再公開の対象外
投稿者停止中	動画処理成功後も自動公開しない
管理者公開再開	投稿者が <code>active</code> の場合だけ許可する

操作

メソッド	事前条件	状態変更	失敗条件・結果
<code>CompleteUpload(now time.Time) error</code>	<code>uploading</code> 、 <code>private</code> 、期限内、元動画存在確認済み	<code>ProcessingStatus</code> を <code>uploaded</code> へ変更する	期限切れ、削除済み、状態不正
<code>ExpireUpload(now time.Time) error</code>	<code>uploading</code> 、 <code>private</code> 、期限到達	<code>ProcessingStatus</code> を <code>expired</code> へ変更する	期限前、状態不正
<code>StartProcessing(now time.Time) error</code>	<code>uploaded</code> 、 <code>private</code> 、未削除	<code>ProcessingStatus</code> を <code>processing</code> へ変更する	削除済み、状態不正
<code>RecordSourceValidation(meta SourceVideoMeta, now time.Time) error</code>	<code>processing</code> 、 <code>SourceMeta</code> 未設定、未削除	<code>SourceMeta</code> を設定する	動画条件違反、二重設定、状態不正
<code>CompleteProcessing(meta OutputVideoMeta, ownerActive bool, now time.Time) error</code>	<code>processing</code> 、 <code>SourceMeta</code> 設定済み、未削除	<code>ready</code> へ変更する。投稿者が <code>active</code> なら <code>published</code> 、それ以外は <code>private</code>	出力条件違反、状態不正
<code>FailProcessing(now time.Time) error</code>	<code>processing</code> 、未削除	<code>failed</code> 、 <code>private</code> へ変更する	状態不正
<code>SetPrivateByOwner(userID uint64, now time.Time) error</code>	投稿者本人、 <code>ready</code> 、 <code>published</code>	<code>private</code> へ変更する	他人、削除済み、状態不正
<code>RepublishByOwner(userID uint64, ownerActive bool, now time.Time) error</code>	投稿者本人、 <code>ready</code> 、 <code>private</code> 、投稿者 <code>active</code>	<code>published</code> へ変更する	他人、停止中、 <code>hidden</code> 、削除済み
<code>HideByAdmin(now time.Time) error</code>	<code>ready</code> 、 <code>published</code> 、未削除	<code>hidden</code> へ変更する	状態不正、削除済み
<code>RestoreByAdmin(ownerActive bool, now time.Time) error</code>	<code>ready</code> 、 <code>hidden</code> 、投稿者 <code>active</code>	<code>published</code> へ変更する	投稿者停止中、状態不正
<code>DeleteByOwner(userID uint64, now time.Time) error</code>	投稿者本人、未削除	<code>DeletedAt</code> を設定し、 <code>private</code> へ変更する	他人、削除済み
<code>CanBeViewedPublicly() bool</code>	なし	なし	公開条件を満たす場合だけ

メソッド	事前条件	状態変更	失敗条件・結果
			true
CanReceiveReaction() bool	なし	なし	公開条件を満たす場合だけ true
IsOwnedBy(userID uint64) bool	なし	なし	UserID が一致する場合だけ true

状態遷移

現在状態	操作	次状態	条件
uploading / private	アップロード完了	uploaded / private	期限内、元動画あり
uploading / private	期限切れ	expired / private	期限到達
uploaded / private	処理開始	processing / private	ジョブ排他取得済み
processing / private	検証成功	processing / private	SourceMeta 設定
processing / private	処理成功	ready / published	投稿者 active
processing / private	処理成功	ready / private	投稿者 suspended
processing / private	最終失敗	failed / private	再試行終了
ready / published	投稿者非公開	ready / private	投稿者本人
ready / private	投稿者再公開	ready / published	投稿者本人かつ active
ready / published	管理者非公開	ready / hidden	管理者操作
ready / hidden	管理者公開再開	ready / published	投稿者 active
任意の未削除状態	投稿削除	private + 論理削除	投稿者本人

動画形式の正規化

ブラウザが送信したファイル名、拡張子、MIMEタイプだけでは動画形式を確定しない。

ストレージ上の実ファイルをFFprobeで解析し、内部の VideoContainer へ正規化する。

入力情報の優先順位

優先順位	情報	扱い
1	FFprobeの実ファイル解析結果	最優先
2	サーバー側で判定したMIMEタイプ	補助判定

優先順位	情報	扱い
3	ブラウザ申告MIMEタイプ	参考情報
4	ファイル拡張子	参考情報

許可する形式

内部値	許可する実形式	正規化後MIMEタイプ
mp4	ISO Base Media、MP4系コンテナ	video/mp4
mov	QuickTime MOV系コンテナ	video/quicktime

FFprobe結果の正規化

FFprobeの `format_name` は複数の名前を返す場合がある。そのため、生の文字列をそのままEntityへ保存せず、Workerで内部値へ変換する。

WorkerはFFprobeの `format_name` および `major_brand` から、`mp4` または `mov` へ正規化する。判定不能な値を推測でどちらかへ割り当てず、エラーとして拒否する。

条件	正規化結果
<code>major_brand = qt</code>	mov
MP4系の <code>major_brand</code>	mp4
format_nameがISO Base Media系だがbrand判定不能	拒否
QuickTime・MP4以外	拒否

MIMEタイプの扱い

条件	処理
Containerがmp4、MIMEがvideo/mp4	許可
Containerがmov、MIMEがvideo/quicktime	許可
ブラウザ申告MIMEだけ不一致	実形式を優先し、セキュリティログへ記録可能
実形式が許可外	拒否
FFprobeで解析不能	拒否
映像トラックなし	拒否
拡張子だけ許可形式	許可根拠にしない

正規化処理の責務

処理	担当
FFprobe実行	Worker
FFprobe出力解析	Worker
Container正規化	Worker
MIME正規化	Worker
SourceVideoMeta生成	Entity Constructor
条件外エラー	Domain Error
DB保存	Repository

SourceVideoMeta

FFprobeで確認した入力動画の実ファイル情報を保持する。

フィールド

フィールド	型	仕様
<code>MIMETYPE</code>	<code>string</code>	ストレージから取得した元動画をサーバー側で検査し、実ファイルから判定したMIMEタイプを保存する。ブラウザから申告された値だけを信用しない
<code>Container</code>	<code>VideoContainer</code>	FFprobeで判定した実際のコンテナ形式を保存する。 <code>mp4</code> または <code>mov</code> だけを許可する。ファイル名、拡張子、ブラウザ申告MIMEタイプではなく、実ファイルの解析結果を優先する
<code>SizeBytes</code>	<code>int64</code>	ストレージ上に保存された元動画の実ファイルサイズをbyte単位で保存する。30MB以下の判定に使用する
<code>DurationMillis</code>	<code>int64</code>	FFprobeで取得した動画の再生時間をミリ秒単位で保存する。10,000ミリ秒以下の判定に使用する
<code>Width</code>	<code>int</code>	動画の回転情報を適用した後の実際の横幅をpixel単位で保存する。入力最大解像度と9:16のアスペクト比判定に使用する
<code>Height</code>	<code>int</code>	動画の回転情報を適用した後の実際の縦幅をpixel単位で保存する。入力最大解像度と9:16のアスペクト比判定に使用する
<code>FrameRate</code>	<code>float64</code>	FFprobeで取得した入力動画のフレームレートを保存する。60fps以下であることを確認するために使用する
<code>VideoCodec</code>	<code>string</code>	入力動画の映像コーデック名を保存する。入力動画の調査、障害原因の特定、変換処理の判断に使用する

フィールド	型	仕様
<code>HasAudio</code>	<code>bool</code>	元動画に音声トラックが存在するかを表す。音声トラックが存在する場合は <code>true</code> 、存在しない場合は <code>false</code> とする
<code>AudioCodec</code>	<code>string</code>	元動画に音声トラックが存在する場合、その音声コーデック名を保存する。 <code>HasAudio</code> が <code>false</code> の場合は空文字とする

生成方法

SourceVideoMetaは、WorkerがFFprobeの解析結果を正規化し、動画条件を満たすことを確認した後に生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewSourceVideoMeta(...)</code> <code>(*SourceVideoMeta, error)</code>	FFprobeから取得・正規化した実ファイル情報	検証済みの入力動画情報	容量、時間、形式、解像度、比率、fps、映像トラック条件に違反

```
NewSourceVideoMeta(
    mimeType string,
    container VideoContainer,
    sizeBytes int64,
    durationMillis int64,
    width int,
    height int,
    frameRate float64,
    videoCodec string,
    hasAudio bool,
    audioCodec string,
) (*SourceVideoMeta, error)
```

不変条件

項目	条件
サイズ	0より大きく30MB以下
時間	0より大きく10秒以下
コンテナ	<code>mp4</code> または <code>mov</code>
最大解像度	幅1080以下、高さ1920以下
アスペクト比	回転情報適用後に9:16

項目	条件
フレームレート	0より大きく60fps以下
映像トラック	必須
音声トラック	任意
<code>HasAudio = false</code>	<code>AudioCodec</code> は空文字
実体検証	FFprobeで解析可能

アスペクト比は浮動小数点ではなく、次の整数式で判定。

`Width × 16 = Height × 9`

OutputVideoMeta

動画変換とサムネイル生成に成功した結果を保持する

フィールド

フィールド	型	仕様
<code>VideoObjectKey</code>	<code>string</code>	変換後動画をオブジェクトストレージ内で特定する内部キー。変換成功後にWorkerが設定する。署名付きURLではなく、期限を持たないストレージキーを保存する
<code>ThumbnailObjectKey</code>	<code>string</code>	生成したサムネイル画像をストレージ内で特定する内部キー。サムネイル生成成功後に設定する。動画公開には空文字でないことが必要
<code>Container</code>	<code>VideoContainer</code>	変換後動画のコンテナ形式。Webブラウザでの再生互換性を統一するため、 <code>mp4</code> 固定とする
<code>Width</code>	<code>int</code>	変換後動画の横幅。720pixel固定とし、変換結果が要件を満たすか確認するために使用する
<code>Height</code>	<code>int</code>	変換後動画の縦幅。1280pixel固定とし、変換結果が要件を満たすか確認するために使用する
<code>FrameRate</code>	<code>float64</code>	変換後動画のフレームレート。30fps以下であることを確認し、端末負荷と通信量を抑える
<code>VideoCodec</code>	<code>string</code>	変換後動画の映像コーデック。ブラウザ互換性を統一するため、 <code>h264</code> 固定とする
<code>HasAudio</code>	<code>bool</code>	変換後動画に音声トラックが存在するかを表す。元動画が無音の場合は <code>false</code> を許可する
<code>AudioCodec</code>	<code>string</code>	変換後動画に音声トラックがある場合は <code>aac</code> を保存する。 <code>HasAudio</code> が <code>false</code> の場合は空文字とする

生成方法

OutputVideoMetaは、Workerが変換後動画とサムネイルの存在・形式を確認した後に生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewOutputVideoMeta(...)</code> <code>(*OutputVideoMeta, error)</code>	変換後動画キー、サムネイルキー、変換後メタデータ	公開可能な動画出力情報	キー、形式、解像度、fps、映像・音声コーデックが不正

```
NewOutputVideoMeta(  
    videoObjectKey string,  
    thumbnailObjectKey string,  
    container VideoContainer,  
    width int,  
    height int,  
    frameRate float64,  
    videoCodec string,  
    hasAudio bool,  
    audioCodec string,  
) (*OutputVideoMeta, error)
```

不変条件

項目	条件
<code>VideoObjectKey</code>	空文字ではない
<code>ThumbnailObjectKey</code>	空文字ではない
<code>Container</code>	<code>mp4</code>
<code>VideoCodec</code>	<code>h264</code>
解像度	720×1280
<code>FrameRate</code>	30fps以下
<code>HasAudio = true</code>	<code>AudioCodec</code> は <code>aac</code>
<code>HasAudio = false</code>	<code>AudioCodec</code> は空文字
無音動画	映像のみのMP4として公開可能

VideoProcessingJob

1動画につき1件の動画検証・変換ジョブを管理する。
複数Workerによる重複実行、無限再試行、処理結果の消失を防ぐ。

VideoJobStatus

値	内容
<code>queued</code>	実行待ち
<code>running</code>	実行中
<code>retry_wait</code>	再試行待ち
<code>succeeded</code>	処理成功
<code>failed</code>	最終失敗
<code>cancelled</code>	取消済み

試行回数の定義

要件の「再試行最大3回」は、初回実行とは別に再実行できる回数を表す。

項目	回数
初回実行	1回
最大再試行回数	3回
最大実行回数	4回
<code>MaxAttempts</code>	4

AttemptCountの数え方

タイミング	AttemptCount
ジョブ作成時	0
初回のStart実行後	1
1回目の再試行開始後	2
2回目の再試行開始後	3
3回目の再試行開始後	4

再試行可否

条件	処理
<code>AttemptCount < MaxAttempts</code> かつ一時的障害	<code>retry_wait</code> へ変更
<code>AttemptCount >= MaxAttempts</code>	<code>failed</code> へ変更

条件	処理
恒久的障害	残回数に関係なく <code>failed</code> へ変更
処理成功	<code>succeeded</code> へ変更

一時的障害

例	再試行
ストレージの一時的な通信失敗	○
外部サービスの一時的な5xx	○
一時的なネットワーク切断	○
Workerの一時的なリソース不足	○
動画形式不正	×
動画破損	×
動画時間超過	×
ファイルサイズ超過	×
アスペクト比不正	×

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	動画処理ジョブを一意に識別する番号。ジョブ作成時にDBが発行し、Workerログや障害調査でジョブを特定するために使用する
<code>VideoID</code>	<code>uint64</code>	検証、変換、サムネイル生成を行う対象動画のID。1動画につき1件のジョブだけを作成し、DBのUNIQUE制約で重複作成を防ぐ
<code>Status</code>	<code>VideoJobStatus</code>	Worker処理の現在状態。作成時は <code>queued</code> 、Worker取得時は <code>running</code> 、一時的障害時は <code>retry_wait</code> 、成功時は <code>succeeded</code> へ変更する
<code>AttemptCount</code>	<code>int</code>	Workerがこのジョブの処理を開始した回数。ジョブ作成時は0とし、 <code>Start()</code> で <code>running</code> へ変更するたびに1増加する
<code>MaxAttempts</code>	<code>int</code>	初回実行を含めて許可する最大実行回数。初回1回と再試行3回を合わせて4を保存する
<code>AvailableAt</code>	<code>time.Time</code>	Workerがこのジョブを次に取得してよい日時。初回はジョブ作成日時を設定し、再試行時は待機時間を加算した日時へ更新する

フィールド	型	仕様
<code>StartedAt</code>	<code>*time.Time</code>	Workerが最後に処理を開始した日時。まだ一度も実行していない場合は <code>nil</code> とする。再試行時は新しい開始日時へ更新する
<code>FinishedAt</code>	<code>*time.Time</code>	ジョブが成功、最終失敗、取消のいずれかで終了した日時。 <code>queued</code> 、 <code>running</code> 、 <code>retry_wait</code> では <code>nil</code> とする
<code>LastErrorCode</code>	<code>string</code>	最後に発生したエラーを機械的に分類する安全なコード。 <code>storage_temporary_error</code> 、 <code>processing_timeout</code> 、 <code>invalid_video_format</code> など、処理分岐と障害集計に使用できる値を保存する
<code>LastErrorMessage</code>	<code>string</code>	障害調査のために最後の失敗内容を保存する。SQL、スタックトレース、秘密鍵、署名URL、認証情報などは含めない
<code>CreatedAt</code>	<code>time.Time</code>	動画処理ジョブを作成した日時。アップロード完了処理で設定し、その後は変更しない
<code>UpdatedAt</code>	<code>time.Time</code>	<code>Status</code> 、 <code>AttemptCount</code> 、 <code>AvailableAt</code> 、開始・終了日時、エラー情報などを最後に変更した日時

生成方法

VideoProcessingJobは、アップロード完了処理で1動画につき1件だけ生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewVideoProcessingJob(videoID uint64, now time.Time) (*VideoProcessingJob, error)</code>	対象動画ID、作成日時	Statusを <code>queued</code> 、AttemptCountを 0、MaxAttemptsを 4、AvailableAtを nowへ設定する	VideoIDが0

不変条件

項目	条件
<code>VideoID</code>	同じVideoIDのジョブは1件
<code>AttemptCount</code>	<code>MaxAttempts</code> を超えない
<code>running</code>	遷移時に <code>AttemptCount</code> を1増加する
再試行	一時的障害だけを対象とする
恒久的障害	ファイル形式不正、破損、上限超過は再試行しない

項目	条件
終端状態	<code>succeeded</code> 、 <code>failed</code> 、 <code>cancelled</code>
エラー情報	SQL、秘密情報、スタックトレースを含めない
タイムアウト	5分の実行制御はWorkerが行う

runningジョブの異常回収

WorkerがJobを `running` へ変更した後、処理完了前に停止すると、Jobが `running` のまま残る。

そのため、一定時間を超えて完了していない `running` ジョブを回収する。

異常判定

項目	条件
対象状態	<code>running</code>
StartedAt	<code>nil</code> ではない
タイムアウト	<code>StartedAt + 5分 <= now</code>
回収処理	定期実行UsecaseまたはWorkerが行う
排他制御	対象Jobを行ロックしてから状態変更する

回収後の状態

条件	Jobの次状態	Videoの状態
AttemptCountがMaxAttempts未満	<code>retry_wait</code>	<code>processing</code> を維持
AttemptCountがMaxAttempts以上	<code>failed</code>	<code>failed / private</code>
動画が削除済み	<code>cancelled</code>	削除状態を維持

操作

メソッド	事前条件	状態変更	失敗条件・結果
<code>Start(now time.Time)</code> <code>error</code>	<code>queued</code> または実行時刻到達済み <code>retry_wait</code> 、試行上限未到達	<code>running</code> へ変更し、 <code>AttemptCount</code> を増加する	実行時刻前、上限到達、状態不正
<code>Succeed(now time.Time)</code> <code>error</code>	<code>running</code>	<code>succeeded</code> へ変更し、 <code>FinishedAt</code> を設定する	状態不正

メソッド	事前条件	状態変更	失敗条件・結果
<code>ScheduleRetry(code, message string, nextAt, now time.Time) error</code>	<code>running</code> 、上限未到達、一時的障害	<code>retry_wait</code> へ変更し、 <code>AvailableAt</code> とエラー情報を設定する	上限到達、恒久的障害、状態不正
<code>Fail(code, message string, now time.Time) error</code>	<code>running</code>	<code>failed</code> へ変更し、 <code>FinishedAt</code> を設定する	状態不正
<code>Cancel(now time.Time) error</code>	<code>queued</code> 、 <code>retry_wait</code> 、 <code>running</code>	<code>cancelled</code> へ変更する	終端状態
<code>IsTimedOut(now time.Time, timeout time.Duration) bool</code>	Statusが <code>running</code> 、 <code>StartedAt</code> が設定済み	なし	<code>StartedAt + timeout <= now</code> の場合にtrue
<code>RecoverTimedOut(timeout time.Duration, nextAt time.Time, now time.Time) error</code>	<code>running</code> 、タイムアウト済み、 <code>AttemptCount</code> が <code>MaxAttempts</code> 未満	<code>retry_wait</code> へ変更し、 <code>AvailableAt</code> を <code>nextAt</code> 、 <code>LastErrorCode</code> を <code>processing_timeout</code> 、 <code>UpdatedAt</code> を <code>now</code> へ変更する	未タイムアウト、上限到達、状態不正
<code>FailTimedOut(timeout time.Duration, now time.Time) error</code>	<code>running</code> 、タイムアウト済み、 <code>AttemptCount</code> が <code>MaxAttempts</code> 以上	<code>failed</code> へ変更し、 <code>FinishedAt</code> 、 <code>LastErrorCode</code> 、 <code>UpdatedAt</code> を設定する	未タイムアウト、上限未到達、状態不正

状態遷移

現在状態	操作	次状態	条件
<code>queued</code>	Workerが処理開始	<code>running</code>	実行可能時刻到達、試行上限未到達
<code>retry_wait</code>	Workerが再試行開始	<code>running</code>	実行可能時刻到達、試行上限未到達
<code>running</code>	処理成功	<code>succeeded</code>	動画変換とサムネイル生成成功
<code>running</code>	一時的障害	<code>retry_wait</code>	試行上限未到達
<code>running</code>	恒久的障害	<code>failed</code>	動画形式不正など
<code>running</code>	試行上限到達	<code>failed</code>	<code>AttemptCount >= MaxAttempts</code>
<code>running</code>	タイムアウト回収	<code>retry_wait</code>	試行上限未到達
<code>running</code>	タイムアウト最終失敗	<code>failed</code>	試行上限到達

現在状態	操作	次状態	条件
<code>queued / retry_wait / running</code>	動画削除	<code>cancelled</code>	対象Videoが削除済み

VideoLike

ログインユーザーと公開動画のいいね関係を管理する。

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	いいねレコードを一意に識別する番号。DBが作成時に発行し、削除対象や障害調査時のレコード特定に使用する
<code>UserID</code>	<code>uint64</code>	いいねを行ったログインユーザーのID。ユーザー本人のいいね状態確認と、ユーザー単位の取得に使用する
<code>VideoID</code>	<code>uint64</code>	いいね対象となる公開動画のID。 <code>UserID</code> との組み合わせを一意にし、同じユーザーが同じ動画へ重複登録できないようにする
<code>CreatedAt</code>	<code>time.Time</code>	いいねを登録した日時。いいね作成時に設定し、解除時はレコードを物理削除する

生成方法

VideoLikeは、Usecaseがユーザー状態と動画公開状態を確認した後に生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewVideoLike(userID, videoID uint64, now time.Time) (*VideoLike, error)</code>	操作ユーザーID、対象動画ID、作成日時	UserID、VideoID、CreatedAtを設定する	UserIDまたはVideoIDが0

不変条件

項目	条件
重複	<code>UserID</code> と <code>VideoID</code> の組み合わせは一意
User	<code>active</code> ユーザーだけが操作できる
Video	公開条件を満たす動画だけへ作成できる
解除	物理削除する
非公開後	関係行は保持してよい

項目	条件
一覧表示	非公開・削除済み動画を表示しない

ユーザー状態と動画公開状態は、Usecaseが取得して確認する。

SavedVideo

ログインユーザーと保存動画の関係を管理する。

フィールド

フィールド	型	仕様
ID	uint64	保存レコードを一意に識別する番号。保存作成時にDBが発行し、削除対象や障害調査時のレコード特定に使用する
UserID	uint64	動画を保存したログインユーザーのID。保存一覧をユーザー単位で取得するために使用する
VideoID	uint64	保存対象の動画ID。UserID との組み合わせを一意にし、同じ動画の重複保存を防ぐ
CreatedAt	time.Time	動画を保存した日時。保存一覧の並び順などに使用する。保存解除時はレコードを物理削除する

生成方法

SavedVideoは、Usecaseがユーザー状態と動画公開状態を確認した後に生成する。

メソッド	引数	生成する状態	失敗条件
NewSavedVideo(userID, videoID uint64, now time.Time) (*SavedVideo, error)	保存ユーザーID、対象動画ID、作成日時	UserID、VideoID、CreatedAtを設定する	UserIDまたはVideoIDが0

不変条件

項目	条件
重複	UserID と VideoID の組み合わせは一意
User	active ユーザーだけが操作できる
Video	公開条件を満たす動画だけを新規保存できる
解除	物理削除する
保存一覧	公開条件を満たす動画だけを表示する

IdempotencyRecord

動画投稿開始APIの重複実行を防ぐ。

同じ冪等性キーで再送された場合は、新しいVideoを作成せず、最初に作成したVideoへ紐づける。

IdempotencyScope

値	内容
<code>video_create</code>	動画投稿開始

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	冪等性記録を一意に識別する番号。DBが作成時に発行し、対象レコードの特定に使用する
<code>UserID</code>	<code>uint64</code>	動画投稿開始リクエストを実行したユーザーID。同じ冪等性キーを別ユーザー間で共有しないために使用する
<code>Scope</code>	<code>IdempotencyScope</code>	冪等性キーを適用する操作の種類。現要件では動画投稿開始を表す <code>video_create</code> を保存する
<code>KeyHash</code>	<code>string</code>	リクエストヘッダーで受け取った平文 <code>Idempotency-Key</code> を、専用秘密鍵を使用したHMAC-SHA-256で変換し、小文字の16進数64文字で保存する。平文キーと完全なHash値はログへ出力しない
<code>RequestHash</code>	<code>string</code>	タイトル、説明、カテゴリーなどを定められた順序・形式で正規化し、SHA-256で変換した小文字の16進数64文字を保存する。同じ冪等性キーが同じ入力へ使用されたかを判定する
<code>ResourceID</code>	<code>uint64</code>	最初のリクエストで作成したVideoのID。同じリクエストが再送された場合、新しいVideoを作成せずこのIDを返す
<code>ExpiresAt</code>	<code>time.Time</code>	この冪等性記録を有効として扱う期限。期限内の同一キー再送では既存結果を返し、期限後は定期削除の対象とする
<code>CreatedAt</code>	<code>time.Time</code>	冪等性記録を作成した日時。Video作成と同じDBトランザクション内で設定し、その後は変更しない

生成方法

IdempotencyRecordは、動画投稿開始時にVideoと同じDBトランザクション内で生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewIdempotencyRecord(userID uint64, scope IdempotencyScope, keyHash, requestHash string, resourceID uint64, expiresAt, now time.Time) (*IdempotencyRecord, error)</code>	ユーザーID、対象操作、Key Hash、Request Hash、作成Video ID、有効期限、作成日時	冪等性判定に必要な値を保存する	各IDが0、Hashが空、Scope不正、期限がnow以下

判定

メソッド	結果
<code>IsExpired(now time.Time) bool</code>	<code>now >= ExpiresAt</code> の場合に <code>true</code>
<code>MatchesRequest(requestHash string) bool</code>	保存済みRequestHashと一致する場合に <code>true</code>

不変条件

項目	条件
一意条件	<code>UserID</code> 、 <code>Scope</code> 、 <code>KeyHash</code> の組み合わせを一意にする
同一リクエスト	同一 <code>RequestHash</code> なら既存 <code>ResourceID</code> を返す
異なるリクエスト	同じキーで <code>RequestHash</code> が異なる場合は拒否する
DB整合性	Video作成と同じDBトランザクションで作成する
秘密情報	平文 <code>Idempotency-Key</code> を保存・ログ出力しない
KeyHash	小文字16進数64文字
RequestHash	小文字16進数64文字
正規化	同じ入力から必ず同じRequestHashが生成される
秘密鍵	Refresh Token用HMAC鍵とは別の鍵を使用する

アップロード完了通知の重複は、Video状態、Video行ロック、`VideoProcessingJob.VideoID` の一意制約で防ぐ。

AdminAuditLog

管理者が、誰に対して、何を、なぜ、いつ行ったかを記録する。監査ログは作成後に更新・削除しない。

AuditTargetType

値	内容
<code>user</code>	ユーザー
<code>video</code>	動画

AuditAction

値	内容
<code>user_suspend</code>	ユーザー利用停止
<code>user_resume</code>	ユーザー利用再開
<code>video_hide</code>	動画非公開
<code>video_restore</code>	動画公開再開
<code>video_hide_by_user_suspension</code>	ユーザー停止に伴う動画非公開

フィールド

フィールド	型	仕様
<code>ID</code>	<code>uint64</code>	監査ログを一意に識別する番号。管理操作実行時にDBが発行し、作成後は変更しない
<code>AdminUserID</code>	<code>uint64</code>	管理操作を実行した管理者のユーザーID。誰が操作したかを追跡するために保存する
<code>TargetType</code>	<code>AuditTargetType</code>	操作対象の種類。ユーザーを対象とする場合は <code>user</code> 、動画を対象とする場合は <code>video</code> を保存する
<code>TargetID</code>	<code>uint64</code>	操作対象となったUserまたはVideoのID。対象レコードが後から変更されても、どの対象へ操作したか追跡するために保存する
<code>Action</code>	<code>AuditAction</code>	管理者が実行した操作の種類。利用停止、利用再開、動画非公開、動画公開再開などを識別する
<code>BeforeStatus</code>	<code>string</code>	管理操作を行う直前の対象状態。ユーザー停止前なら <code>active</code> 、動画非公開前なら <code>published</code> などを保存する
<code>AfterStatus</code>	<code>string</code>	管理操作が成功した後の対象状態。ユーザー停止後なら <code>suspended</code> 、動画非公開後なら <code>hidden</code> などを保存する
<code>Reason</code>	<code>string</code>	管理者が入力した操作理由。前後の空白を除去して1～500文字で保存する。パスワード、Token、Cookieなどの秘密情報は含めない
<code>RequestID</code>	<code>string</code>	管理操作を受け付けたHTTPリクエストのRequest ID。APIログと監査ログを紐づけ、障害調査時に同じ処理を追

フィールド	型	仕様
		跡するために使用する
CreatedAt	time.Time	管理操作が成功した日時。監査ログ作成時に設定し、その後は変更しない

生成方法

AdminAuditLogは、管理操作が成功するDBトランザクション内で生成する。

メソッド	引数	生成する状態	失敗条件
NewAdminAuditLog(...) (*AdminAuditLog, error)	管理者ID、対象種別、 対象ID、操作、変更前 後状態、理由、 Request ID、作成日時	更新・削除しない監査記録	IDが0、Actionと TargetType不一致、 状態が同じ、理由不正、RequestIDが空

```
NewAdminAuditLog(
    adminUserID uint64,
    targetType AuditTargetType,
    targetID uint64,
    action AuditAction,
    beforeStatus string,
    afterStatus string,
    reason string,
    requestID string,
    now time.Time,
) (*AdminAuditLog, error)
```

不変条件

項目	条件
Reason	trim後1～500文字
BeforeStatus	AfterStatus と異なる
Action	TargetType との組み合わせが正しい
更新	作成後に更新しない
削除	削除しない
秘密情報	パスワード、Token、Cookie、秘密鍵、署名URLを記録しない
管理者確認	active な管理者かどうかはUsecaseが確認する

Actionと状態変更

Action	TargetType	BeforeStatus	AfterStatus
user_suspend	user	active	suspended
user_resume	user	suspended	active
video_hide	video	published	hidden
video_restore	video	hidden	published
video_hide_by_user_suspension	video	published	hidden

StorageCleanupJob

動画削除、アップロード期限切れ、動画処理失敗、補償処理、孤立ファイル検出によって不要になったファイルの削除状態を管理する。

CleanupJobStatus

値	内容
queued	実行待ち
running	実行中
retry_wait	再試行待ち
succeeded	削除成功
failed	最終失敗

StorageAssetType

値	内容
original	元動画
processed	変換後動画
thumbnail	サムネイル
unknown	種別不明の孤立ファイル

CleanupCause

値	内容
video_delete	投稿削除
upload_expired	アップロード期限切れ

値	内容
<code>process_failed</code>	動画処理失敗
<code>rollback_cleanup</code>	DB更新失敗後の補償
<code>orphan_detected</code>	孤立ファイル検出

試行回数の定義

要件の「再試行最大3回」は、初回実行とは別に再実行できる回数を表す。

項目	回数
初回実行	1回
最大再試行回数	3回
最大実行回数	4回
<code>MaxAttempts</code>	4

AttemptCountの数え方

タイミング	AttemptCount
ジョブ作成時	0
初回のStart実行後	1
1回目の再試行開始後	2
2回目の再試行開始後	3
3回目の再試行開始後	4

再試行可否

条件	処理
<code>AttemptCount < MaxAttempts</code> かつ一時的障害	<code>retry_wait</code> へ変更
<code>AttemptCount >= MaxAttempts</code>	<code>failed</code> へ変更
恒久的障害	残回数に関係なく <code>failed</code> へ変更
処理成功	<code>succeeded</code> へ変更

一時的障害

例	再試行
ストレージの一時的な通信失敗	○

例	再試行
外部サービスの一時的な5xx	○
一時的なネットワーク切断	○
Workerの一時的なリソース不足	○
動画形式不正	×
動画破損	×
動画時間超過	×
ファイルサイズ超過	×
アスペクト比不正	×

フィールド

フィールド	型	仕様
ID	uint64	ストレージ削除ジョブを一意に識別する番号。ジョブ作成時にDBが発行し、Workerログや障害調査に使用する
VideoID	*uint64	削除対象ファイルが特定の動画に属する場合、そのVideo IDを保存する。DBに対応する動画が存在しない孤立ファイルでは <code>nil</code> を許可する
ObjectKey	string	オブジェクトストレージから削除するファイルの内部キー。署名付きURLではなく、ストレージ上の保存場所を特定する期限のないキーを保存する
AssetType	StorageAssetType	削除対象が元動画、変換後動画、サムネイル、種別不明のどれかを表す。削除結果の調査とログ分類に使用する
Cause	CleanupCause	なぜ削除対象になったかを表す。投稿削除、期限切れ、処理失敗、DB更新失敗後の補償、孤立ファイル検出を区別する
Status	CleanupJobStatus	削除処理の現在状態。作成時は <code>queued</code> 、Worker実行中は <code>running</code> 、一時的障害時は <code>retry_wait</code> 、成功時は <code>succeeded</code> へ変更する
AttemptCount	int	Workerが削除処理を開始した回数。ジョブ作成時は0とし、 <code>Start()</code> を実行するたびに1増加する
MaxAttempts	int	初回実行を含む最大実行回数。初回1回と再試行3回を合わせて4を保存する
AvailableAt	time.Time	Workerが次に削除処理を実行してよい日時。初回は作成日時、再試行時は待機時間を加算した日時を設定する

フィールド	型	仕様
<code>StartedAt</code>	<code>*time.Time</code>	Workerが最後に削除処理を開始した日時。未実行の場合は <code>nil</code> とする。再試行時は新しい開始日時へ更新する
<code>FinishedAt</code>	<code>*time.Time</code>	削除成功または最終失敗として終了した日時。実行待ち、実行中、再試行待ちの場合は <code>nil</code> とする
<code>LastErrorCode</code>	<code>string</code>	最後に発生した削除エラーを機械的に分類する安全なコード。ストレージ一時障害、権限エラーなどを区別する
<code>LastErrorMessage</code>	<code>string</code>	削除失敗の調査に必要な説明。ストレージ認証情報、署名URL、秘密鍵、SQL、スタックトレースを含めない
<code>CreatedAt</code>	<code>time.Time</code>	削除ジョブを作成した日時。動画削除などのDB処理と同じトランザクションで設定し、その後は変更しない
<code>UpdatedAt</code>	<code>time.Time</code>	<code>Status</code> 、 <code>AttemptCount</code> 、 <code>AvailableAt</code> 、開始・終了日時、エラー内容などを最後に変更した日時

生成方法

StorageCleanupJobは、ファイル削除が必要になったDB処理と同じトランザクション内で生成する。

メソッド	引数	生成する状態	失敗条件
<code>NewStorageCleanupJob(videoID uint64, objectKey string, assetType StorageAssetType, cause CleanupCause, now time.Time) (*StorageCleanupJob, error)</code>	関連動画ID、削除対象キー、種別、削除理由、作成日時	Statusを <code>queued</code> 、AttemptCountを0、MaxAttemptsを4、AvailableAtをnowへ設定する	ObjectKeyが空、AssetTypeまたはCauseが不正

running削除ジョブの異常回収

項目	条件
対象状態	<code>running</code>
<code>StartedAt</code>	<code>nil</code> ではない
異常判定	<code>StartedAt.Add(timeout) <= now</code>
試行上限未到達	<code>retry_wait</code> へ変更する
試行上限到達	<code>failed</code> へ変更する

項目	条件
排他制御	Repositoryが対象Jobを行ロックしてから回収する

メソッド

```
IsTimedOut(nowtime.Time, timeouttime.Duration) boolRecoverTime
dOut(timeouttime.Duration, nextAttime.Time, nowtime.Time,
) errorFailTimedOut(timeouttime.Duration, nowtime.Time,
) error
```

不変条件

項目	条件
<code>ObjectKey</code>	空文字ではない
重複	同じ <code>ObjectKey</code> の未完了ジョブを重複作成しない
<code>AttemptCount</code>	<code>MaxAttempts</code> を超えない
対象なし	ストレージ上に対象がない場合も成功扱いにする
秘密情報	署名URLやストレージ認証情報を保持しない
DB整合性	動画論理削除と削除ジョブ作成を同じTxで行う
<code>failed</code>	管理者確認一覧の対象にする

操作

メソッド	事前条件	状態変更	失敗条件・結果
<code>Start(now time.Time)</code> <code>error</code>	<code>queued</code> または 実行可能な <code>retry_wait</code>	<code>running</code> へ変更し、 <code>AttemptCount</code> を増 加する	実行時刻前、上限到達、 状態不正
<code>Succeed(now time.Time)</code> <code>error</code>	<code>running</code>	<code>succeeded</code> へ変更す る	状態不正
<code>ScheduleRetry(code, message string, nextAt, now time.Time)</code> <code>error</code>	<code>running</code> 、上限 未到達	<code>retry_wait</code> へ変更 する	上限到達、状態不正
<code>Fail(code, message string, now time.Time)</code> <code>error</code>	<code>running</code>	<code>failed</code> へ変更する	状態不正
<code>IsTimedOut(now time.Time, timeout time.Duration)</code> <code>bool</code>	なし	なし	<code>Status</code> が <code>running</code> 、 <code>StartedAt</code> が設定済み、 かつ

メソッド	事前条件	状態変更	失敗条件・結果
			<code>StartedAt.Add(timeout)</code> <code><= now</code> の場合に <code>true</code>
<code>RecoverTimedOut(timeout time.Duration, nextAt, now time.Time) error</code>	<code>running</code> 、タイムアウト済み、 <code>AttemptCount < MaxAttempts</code>	<code>retry_wait</code> へ変更し、 <code>AvailableAt</code> を <code>nextAt</code> 、 <code>LastErrorCode</code> を <code>cleanup_timeout</code> 、 <code>UpdatedAt</code> を <code>now</code> へ変更する	未タイムアウト、試行上限到達、状態不正
<code>FailTimedOut(timeout time.Duration, now time.Time) error</code>	<code>running</code> 、タイムアウト済み、 <code>AttemptCount >= MaxAttempts</code>	<code>failed</code> へ変更し、 <code>FinishedAt</code> を <code>now</code> 、 <code>LastErrorCode</code> を <code>cleanup_timeout</code> 、 <code>UpdatedAt</code> を <code>now</code> へ変更する	未タイムアウト、試行上限未到達、状態不正

状態遷移

現在状態	操作	次状態	条件
<code>queued</code>	削除開始	<code>running</code>	実行可能時刻到達
<code>retry_wait</code>	再試行開始	<code>running</code>	実行可能時刻到達
<code>running</code>	削除成功	<code>succeeded</code>	対象削除成功、または対象が既に存在しない
<code>running</code>	一時的障害	<code>retry_wait</code>	試行上限未到達
<code>running</code>	恒久的障害	<code>failed</code>	権限エラーなど
<code>running</code>	試行上限到達	<code>failed</code>	<code>AttemptCount >= MaxAttempts</code>
<code>running</code>	タイムアウト回収	<code>retry_wait</code>	試行上限未到達
<code>running</code>	タイムアウト最終失敗	<code>failed</code>	試行上限到達

Enumの検証

型	必要な検証
<code>UserRole</code>	<code>Valid() bool</code>
<code>UserStatus</code>	<code>Valid() bool</code>
<code>VideoProcessingStatus</code>	<code>Valid() bool</code>

型	必要な検証
VideoPublishStatus	Valid() bool
VideoContainer	Valid() bool
VideoJobStatus	Valid() bool
IdempotencyScope	Valid() bool
AuditTargetType	Valid() bool
AuditAction	Valid() bool
CleanupJobStatus	Valid() bool
StorageAssetType	Valid() bool
CleanupCause	Valid() bool

Domain Error

EntityではHTTPステータスや画面表示文言を扱わない。業務上の意味を表すエラーだけを定義する。

エラー	発生条件
ErrInvalidEntityState	Entityの初期状態またはEnum値が不正
ErrInvalidStateTransition	許可されていない状態遷移
ErrInvalidUserName	名前が1～50文字外
ErrInvalidCategoryCode	未定義カテゴリー
ErrInvalidVideoTitle	タイトルが1～100文字外
ErrInvalidVideoDescription	説明が1000文字超過
ErrUserSuspended	停止中ユーザーが認証必須操作を実行
ErrAdminStatusChangeNotAllowed	管理者の状態を変更しようとした
ErrNotOwner	他人の動画を操作しようとした
ErrVideoNotPublic	公開条件を満たさない動画へ操作した
ErrVideoHiddenByAdmin	投稿者が <code>hidden</code> 動画を再公開しようとした
ErrVideoDeleted	削除済み動画を操作しようとした
ErrUploadExpired	期限切れ後にアップロード完了しようとした
ErrUploadNotExpired	期限前に期限切れへ変更しようとした
ErrSourceVideoInvalid	入力動画が投稿条件を満たさない
ErrOutputVideoInvalid	変換後動画が公開条件を満たさない
ErrRetryLimitExceeded	最大実行回数へ到達

エラー	発生条件
<code>ErrJobNotAvailable</code>	実行可能時刻前にジョブを開始しようとした
<code>ErrTokenExpired</code>	Refresh Token期限切れ
<code>ErrTokenUsed</code>	Refresh Token使用済み
<code>ErrTokenRevoked</code>	Refresh Token失効済み
<code>ErrIdempotencyConflict</code>	同じ冪等性キーへ異なる入力を送信
<code>ErrInvalidAuditReason</code>	管理理由が1~500文字外
<code>ErrInvalidUserID</code>	UserIDが0
<code>ErrInvalidVideoID</code>	VideoIDが0
<code>ErrInvalidEmail</code>	Emailが空または254文字超過
<code>ErrInvalidPasswordHash</code>	PasswordHashが空
<code>ErrInvalidTokenHash</code>	Refresh Token Hashが空
<code>ErrInvalidFamilyID</code>	FamilyIDが空
<code>ErrInvalidReplacementTokenID</code>	ReplacedByIDへ0を設定
<code>ErrInvalidObjectKey</code>	ObjectKeyが空
<code>ErrInvalidUploadExpiry</code>	UploadExpiresAtが作成日時以下
<code>ErrInvalidVideoContainer</code>	許可されていない動画形式
<code>ErrVideoFormatMismatch</code>	実形式と正規化条件が一致しない
<code>ErrVideoProbeFailed</code>	FFprobeで解析できない
<code>ErrVideoTrackMissing</code>	映像トラックが存在しない
<code>ErrProcessingTimedOut</code>	動画処理がタイムアウト
<code>ErrInvalidJobStatus</code>	未定義JobStatus
<code>ErrInvalidCleanupStatus</code>	未定義CleanupJobStatus
<code>ErrInvalidCleanupCause</code>	未定義CleanupCause
<code>ErrInvalidAuditAction</code>	ActionとTargetTypeの組み合わせが不正
<code>ErrInvalidRequestID</code>	RequestIDが空